PostgreSQL Scripts Documentation

Project

Community Library Management System

Overview

This document contains the PostgreSQL functions, procedures, and supporting scripts used in the Community Library Management System.

These routines are responsible for handling important business operations such as:

- Creating borrowing transactions
- Returning books
- Calculating fines
- Retrieving available books
- Generating borrowing summaries

The scripts are written based on the current database schema used in the project, including tables such as:

- members
- membershiptypes
- books
- bookcopies
- borrowings
- fines
- finepayments

---

1. create_borrowing Function

Purpose

This function creates a new borrowing transaction for a member and updates the selected book copy status to `Borrowed`.

The due date is automatically calculated based on the member's membership type.

Parameters

Parameter	Description
`p_member_id`	Member requesting the book
`p_barcodeno`	Barcode of the book copy

Main Validations

The function checks:

- Whether the member exists
- Whether the book copy exists
- Whether the copy is currently available
- Whether membership configuration exists

Operations Performed

- Calculates due date using membership type rules
- Inserts a new borrowing record
- Updates the book copy status

Return Value

Returns the generated borrowing transaction ID.

---

2. return_book Procedure

Purpose

This procedure handles the return process for borrowed books.

It updates:

- borrowing status
- return date

- book copy availability

It also creates a late fine if the book is returned after the due date.

Parameters

Parameter	Description
`p_barcodeno`	Barcode of the borrowed copy
`p_returndate`	Return date (defaults to current date)

Operations Performed

- Finds the active borrowing record
- Marks borrowing as returned
- Updates copy status to `Available`
- Calculates delayed days
- Creates a late return fine when applicable

Fine Calculation Rule

Late return fine:

- ₹10 per delayed day

---

3. calculate_member_fine Function

Purpose

Returns the total unpaid fine amount for a member.

Parameter

Parameter	Description
`p_member_id`	Member ID

Result

Returns:

- Total unpaid fine amount
- Returns `0` if no pending fines exist

---

4. get_available_books_by_category Function

Purpose

Retrieves all available books for a selected category.

The function also returns the number of available copies for each book.

Parameter

Parameter	Description
`p_category_id`	Category ID

Returned Data

- ISBN
- Book title
- Author name
- Available copy count

---

5. get_member_borrowing_summary Function

Purpose

Provides a borrowing summary for a member.

Parameter

Parameter	Description
`p_member_id`	Member ID

Returned Information

- Active borrow count
- Returned borrow count
- Total unpaid fine amount

---

6. Optional Performance Indexes

Additional indexes are added to improve query performance for frequently used operations.

Recommended Indexes

Borrowing Indexes

- memberid + borrowstatus
- barcodeno + borrowstatus

Fine Indexes

- ispaid

Book Indexes

- categoryid

---

7. Validation Queries

The following queries can be used after creating the routines to verify that everything works correctly.

Check Existing Functions and Procedures

Used to verify routine creation.

Test Member Borrowing Summary

Used to check borrowing statistics for a member.

Test Available Books by Category

Used to verify category-based availability reporting.

---

8. Technical Notes

- The routines are designed to work with the current Entity Framework Core DAL implementation.
- Stored procedures are invoked from the repository layer using raw SQL execution.
- The `return_book` procedure supports both default and custom return dates.
- Late fine calculation logic is centralized inside the database procedure for easier maintenance.
- All routines follow the current relational structure and foreign key constraints of the project database.

Conclusion

These PostgreSQL routines help move important business operations closer to the database layer, improving consistency and reducing repeated logic in the application layer.

The scripts support the core workflows of the library system, including borrowing, returning, fine management, and reporting, while maintaining proper data integrity and transaction handling.